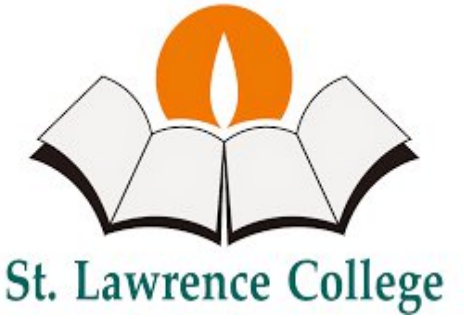
Lab Report of Artificial Intelligence (CSC 266)



Submitted By:

Shudarsan Paudel

BscCSIT 4th Sem

Submitted To:

Deep Raj Bhujel

Table of Contents

LAB 1: Implementing Simple Intelligent Agents

Source Code of Vacuum Cleaner Agent

```
from enum import Enum

class TileStatus(Enum):
    DIRTY = "DIRTY"
    CLEAN = "CLEAN"

class Location(Enum):
    LEFT = "LEFT"
    RIGHT = "RIGHT"

class Action(Enum):
    SUCK_TILE = "SUCK_TILE"
    MOVE_LEFT = "MOVE_LEFT"
    MOVE_RIGHT = "MOVE_RIGHT"

def vacuum_agent(location: Location, status: TileStatus) -> Action:
    if status == TileStatus.DIRTY:
        return Action.SUCK_TILE
    if location == Location.LEFT:
        return Action.MOVE_RIGHT
    elif location == Location.RIGHT:
        return Action.MOVE_LEFT
    else:
        raise ValueError("Invalid Location")

# --- Testing and printing outputs ---
print("LEFT + DIRTY  →", vacuum_agent(Location.LEFT, TileStatus.DIRTY))
print("LEFT + CLEAN  →", vacuum_agent(Location.LEFT, TileStatus.CLEAN))
print("RIGHT + CLEAN →", vacuum_agent(Location.RIGHT, TileStatus.CLEAN))
print("RIGHT + DIRTY →", vacuum_agent(Location.RIGHT, TileStatus.DIRTY))
```

Output:

```
By Shudarsan Paudel
LEFT + DIRTY  → Action.SUCK_TILE
LEFT + CLEAN  → Action.MOVE_RIGHT
RIGHT + CLEAN → Action.MOVE_LEFT
RIGHT + DIRTY → Action.SUCK_TILE
```

Source Code of Traffic Light Agent:

```
from enum import Enum

class TrafficStates(Enum):
RED = 1
GREEN = 2
YELLOW = 3

def traffic_light(current_state: TrafficStates) -> TrafficStates:
match current_state:
case TrafficStates.RED:
return TrafficStates.GREEN
case TrafficStates.GREEN:
return TrafficStates.YELLOW
case TrafficStates.YELLOW:
return TrafficStates.RED
case _:
raise ValueError("Undefined State")

# --- Testing the Traffic Light Agent ---
print("RED ->", traffic_light(TrafficStates.RED))
print("GREEN ->", traffic_light(TrafficStates.GREEN))
print("YELLOW ->", traffic_light(TrafficStates.YELLOW))
```

Output:

```
By Shudarsan Paudel
RED    -> TrafficStates.GREEN
GREEN  -> TrafficStates.YELLOW
YELLOW -> TrafficStates.RED
```

LAB 2: Implementing Various Search Algorithms

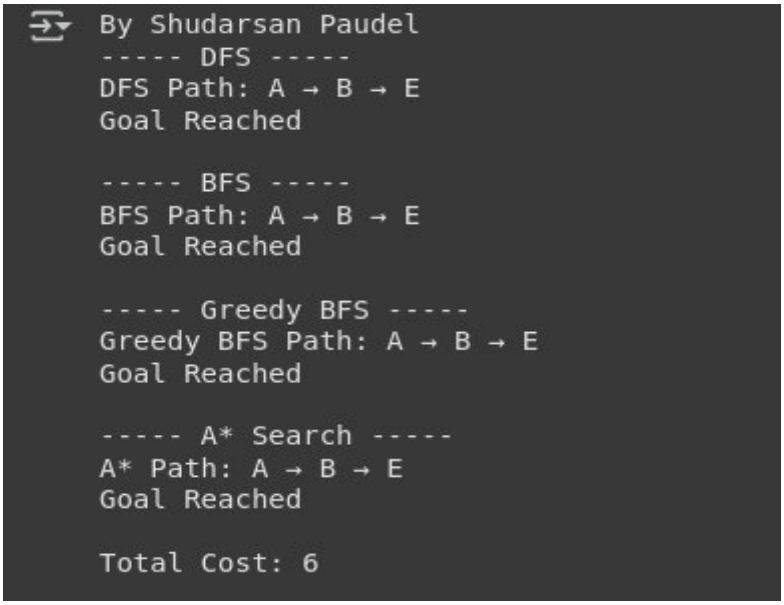
```
# Search Algorithms: BFS, DFS, Greedy BFS, A*

from typing import List, Dict, Tuple
from queue import PriorityQueue


# Node class for BFS/DFS/Greedy
# ----------------------------
class Node:
def __init__(self, value: str):
self.value = value
self.children: List['Node'] = []

def add_child(self, child_node: 'Node'):
self.children.append(child_node)
```

```
# Depth-First Search (DFS)
# ----------------------------
def dfs(start_node: Node, goal: str):
stack = [(start_node, [start_node.value])]
visited = set()

while stack:
node, path = stack.pop()
if node.value == goal:
print("DFS Path:", " → ".join(path))
print("Goal Reached\n")
return
if node not in visited:
visited.add(node)
for child in reversed(node.children):
stack.append((child, path + [child.value]))

# ----------------------------
# Breadth-First Search (BFS)
# ----------------------------
def bfs(start_node: Node, goal: str):
queue = [(start_node, [start_node.value])]
visited = set()
while queue:
node, path = queue.pop(0)
if node.value == goal:
print("BFS Path:", " → ".join(path))
print("Goal Reached\n")
return
if node not in visited:
visited.add(node)
for child in node.children:
queue.append((child, path + [child.value]))

# ----------------------------
# Greedy Best-First Search
# ----------------------------
def greedy_bfs(start_node: Node, goal: str, heuristic: Dict[str, int]):
pq = PriorityQueue()
pq.put((heuristic[start_node.value], start_node, [start_node.value]))
visited = set()

while not pq.empty():
_, node, path = pq.get()
if node.value == goal:
print("Greedy BFS Path:", " → ".join(path))
print("Goal Reached\n")
return
if node not in visited:
visited.add(node)
for child in node.children:
if child not in visited:
pq.put((heuristic[child.value], child, path + [child.value]))
```

```
# A* Search
# ----------------------------
def a_star(start_node: Node, goal: str, heuristic: Dict[str, int], cost:
Dict[Tuple[str,str], int]):
pq = PriorityQueue()
pq.put((heuristic[start_node.value], 0, start_node, [start_node.value]))
visited = {}

while not pq.empty():
f, g, node, path = pq.get()
if node.value == goal:
print("A* Path:", " → ".join(path))
print("Goal Reached\n")
print("Total Cost:", g)
return
if node in visited and visited[node] <= g:
continue
visited[node] = g
for child in node.children:
step_cost = cost.get((node.value, child.value), 1)
g_new = g + step_cost
f_new = g_new + heuristic[child.value]
pq.put((f_new, g_new, child, path + [child.value]))

# ----------------------------
# Example Graph for all searches
# ----------------------------
# Create nodes
A = Node("A")
B = Node("B")
C = Node("C")
D = Node("D")
E = Node("E")
F = Node("F")
G = Node("G")

# Build tree connections
A.add_child(B)
A.add_child(C)
B.add_child(D)
B.add_child(E)
C.add_child(F)
C.add_child(G)

# Heuristic for Greedy BFS and A* (lower = closer to goal E)
heuristic = {"A": 6, "B": 4, "C": 5, "D": 3, "E": 0, "F": 6, "G": 7}
# Costs for A* (edge weights)
cost = {("A","B"):1, ("A","C"):2, ("B","D"):2, ("B","E"):5, ("C","F"):2,
("C","G"):3}

# ----------------------------
# Run all searches serially
# ----------------------------
print(“By Shudarsan Paudel”)
print("----- DFS -----")
```

```
dfs(A, "E")
print("----- BFS -----")
bfs(A, "E")
print("----- Greedy BFS -----")
greedy_bfs(A, "E", heuristic)
print("----- A* Search -----")
a_star(A, "E", heuristic, cost)
```

Output:

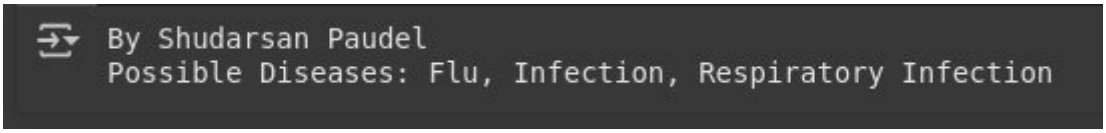


LAB 3: Implementing Knowledge Based Questions

```
# Rule-Based System: Find all possible diseases

def diagnose_disease(symptoms):
disease_rules = {
'Flu': {'fever', 'cough'},
'Migraine': {'fever', 'headache'},
'Common Cold': {'cough', 'headache'},
'Infection': {'fever'},
'Respiratory Infection': {'cough'},
'Tension Headache': {'headache'}
}

symptom_set = set(symptoms)
possible_diseases = []

# Check all diseases
for disease, required_symptoms in disease_rules.items():
if required_symptoms.issubset(symptom_set):
possible_diseases.append(disease)

# Print output
print("By Shudarsan Paudel")
if possible_diseases:
```

```
print("Possible Diseases:", ", ".join(possible_diseases))
else:
print("Diagnosis Uncertain")

# Test

symptoms = ["fever", "cold", "cough"]
diagnose_disease(symptoms)
```

Output:



#

Predicate-Based System

```
class PredicateKnowledgeBase:
def __init__(self):
self.facts = set()
self.rules = []

def add_fact(self, fact: str):
self.facts.add(fact)
def add_rule(self, premise: str, conclusion: str):
self.rules.append((premise, conclusion))
def infer(self, query: str) -> bool:
"""
Infers new facts based on rules and prints inference steps.
Returns True if the query can be inferred.
"""
changed = True
while changed:
changed = False
for premise, conclusion in self.rules:
if premise in self.facts and conclusion not in self.facts:
self.add_fact(conclusion)
print(f"Inferred: '{conclusion}' based on '{premise}'")
changed = True # continue until no new facts

return query in self.facts

# Example Usage
kb = PredicateKnowledgeBase()

# Add facts
kb.add_fact("Alice is a parent of Bob")
kb.add_fact("Bob is a parent of Charlie")

# Add rules
kb.add_rule("Alice is a parent of Bob", "Bob is a child of Alice")
kb.add_rule("Bob is a parent of Charlie", "Charlie is a grandchild of Alice")

# Query
```

```
print(“Shudarsan Paudel”)
query = "Charlie is a grandchild of Alice"
if kb.infer(query):
print(f"\n{query} is known.")
else:
print(f"\n{query} is not known.")
```

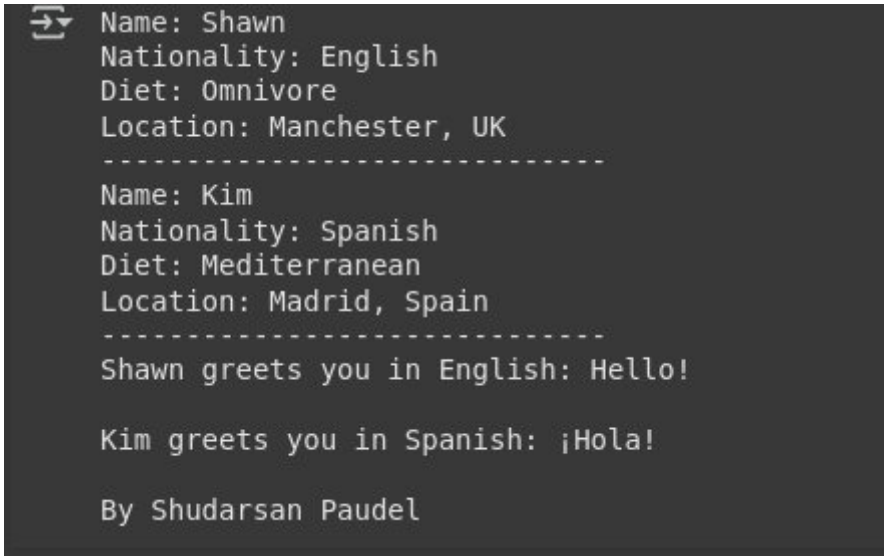
Output of Predicate-Based System (like Prolog):

```
By Shudarsan Paudel
Inferred: 'Bob is a child of Alice' based on 'Alice is a parent of Bob'
Inferred: 'Charlie is a grandchild of Alice' based on 'Bob is a parent of Charlie'

Charlie is a grandchild of Alice is known.
```

```
# Frame-Based System Example

class Frame:
def __init__(self, name, nationality, diet, location):
self.name = name
self.nationality = nationality
self.diet = diet
self.location = location

def display(self):
print(f"Name: {self.name}")
print(f"Nationality: {self.nationality}")
print(f"Diet: {self.diet}")
print(f"Location: {self.location}")
print("-" * 30)

class PersonFrame(Frame):
def __init__(self, name, nationality, diet, location, language, greeting):
super().__init__(name, nationality, diet, location)
self.language = language
self.greeting = greeting

def say_hello(self):
print(f"{self.name} greets you in {self.language}: {self.greeting}")
print()




# ----------------------------
# Create Frame Instances
# ----------------------------
shawn = PersonFrame(
name="Shawn",
nationality="English",
diet="Omnivore",
location="Manchester, UK",
language="English",
```

```
greeting="Hello!"
)

kim = PersonFrame(
name="Kim",
nationality="Spanish",
diet="Mediterranean",
location="Madrid, Spain",
language="Spanish",
greeting="¡Hola!"
)

# ----------------------------
# Display Information
# ----------------------------
shawn.display()
kim.display()

# Greetings
shawn.say_hello()
kim.say_hello()

# Lab Credit
print("By Shudarsan Paudel")
```

Output:

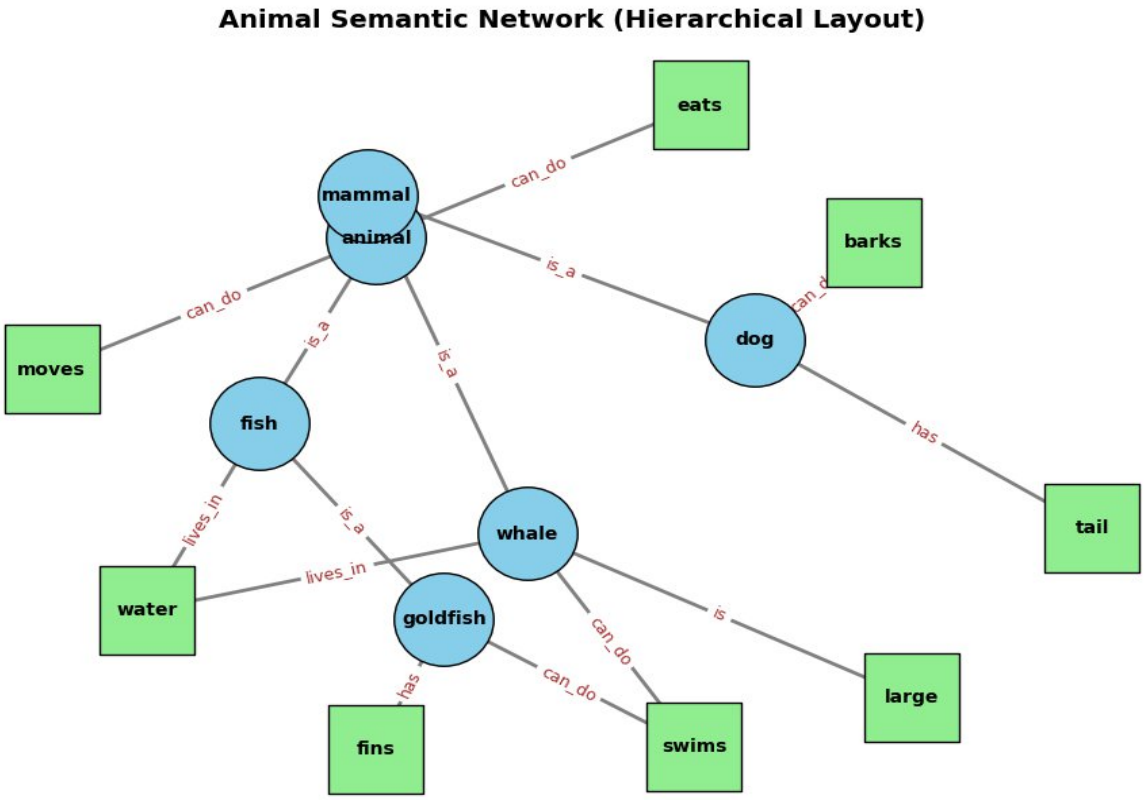



```
# Semantic Network

import networkx as nx
import matplotlib.pyplot as plt

def create_animal_semantic_network():
G = nx.DiGraph()
```

```
# Concept nodes
concepts = ["animal", "mammal", "fish", "dog", "whale", "goldfish"]
# Properties/attributes/actions
attributes = ["tail", "fins", "barks", "swims", "large", "water", "eats",
"moves"]

G.add_nodes_from(concepts, type="concept")
G.add_nodes_from(attributes, type="attribute")

# Relationships
G.add_edge("mammal", "animal", label="is_a")
G.add_edge("fish", "animal", label="is_a")
G.add_edge("dog", "mammal", label="is_a")
G.add_edge("whale", "mammal", label="is_a")
G.add_edge("goldfish", "fish", label="is_a")

G.add_edge("animal", "eats", label="can_do")
G.add_edge("animal", "moves", label="can_do")
G.add_edge("dog", "barks", label="can_do")
G.add_edge("dog", "tail", label="has")
G.add_edge("whale", "large", label="is")
G.add_edge("whale", "swims", label="can_do")
G.add_edge("whale", "water", label="lives_in")
G.add_edge("goldfish", "swims", label="can_do")
G.add_edge("goldfish", "fins", label="has")
G.add_edge("fish", "water", label="lives_in")

return G

# Create semantic network
G = create_animal_semantic_network()

# Try to use graphviz_layout for a hierarchical view (requires pygraphviz or py-
dot)
try:
pos = nx.nx_agraph.graphviz_layout(G, prog="dot") # clean hierarchy
except:
pos = nx.spring_layout(G, seed=42) # fallback if graphviz not available

# Separate node types
concept_nodes = [n for n, d in G.nodes(data=True) if d['type'] == "concept"]
attribute_nodes = [n for n, d in G.nodes(data=True) if d['type'] == "attribute"]

plt.figure(figsize=(12, 8))

# Draw concept nodes (blue circles)
nx.draw_networkx_nodes(G, pos, nodelist=concept_nodes, node_color="skyblue",
node_size=2800, edgecolors="black")
# Draw attribute nodes (green squares)
nx.draw_networkx_nodes(G, pos, nodelist=attribute_nodes, node_color="light-
green", node_size=2500, edgecolors="black", node_shape="s")

# Draw edges
nx.draw_networkx_edges(G, pos, arrowstyle="->", arrowsize=18, edge_color="gray",
width=2)
# Node labels
```

```
nx.draw_networkx_labels(G, pos, font_size=10, font_weight="bold")

# Edge labels
edge_labels = nx.get_edge_attributes(G, 'label')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=9,
font_color="brown")

plt.title("Animal Semantic Network (Hierarchical Layout)", fontsize=14,
fontweight="bold")
plt.axis("off")
plt.show()
```

Output:



LAB 4: Machine Learning

```
# Naive Bayes
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB

# Dataset
data = {
"Outlook": ["Sunny", "Overcast", "Rainy", "Sunny", "Sunny", "Overcast",
"Rainy"],
"Temp": ["Hot", "Hot", "Mild", "Cool", "Hot", "Mild", "Cool"],
```

```
"Humidity": ["High", "High", "High", "Normal", "Normal", "Normal", "High"],
"Windy": ["False", "True", "False", "False", "True", "True", "False"],
"Play": ["No", "Yes", "Yes", "Yes", "No", "Yes", "No"]
}
df = pd.DataFrame(data)
# Encode categorical features
encoders = {col: LabelEncoder() for col in df.columns if col != "Play"}
encoded_df = df.copy()
for col, encoder in encoders.items():
encoded_df[col] = encoder.fit_transform(df[col])
X = encoded_df.drop("Play", axis=1)
y = df["Play"]

# Train Naive Bayes model
classifier = GaussianNB()
classifier.fit(X, y)
print("By Shudarsan Paudel")
print("GaussianNB Model Trained")

# Predictions
y_pred = classifier.predict(X)
print("Predictions:", y_pred)

# Show Actual vs Predicted
print("\nActual vs Predicted:")
for actual, pred in zip(y, y_pred):
print(f"Actual: {actual:3} -> Predicted: {pred}")
```

Output of Native Bayes:

```
By Shudarsan Paudel
GaussianNB Model Trained
Predictions: ['No' 'Yes' 'Yes' 'No' 'No' 'Yes' 'No']

Actual vs Predicted:
Actual: No  -> Predicted: No
Actual: Yes -> Predicted: Yes
Actual: Yes -> Predicted: Yes
Actual: Yes -> Predicted: No
Actual: No  -> Predicted: No
Actual: Yes -> Predicted: Yes
Actual: No  -> Predicted: No
```

 # Neural Network for realization of AND & OR Gates

```
import numpy as np

def sigmoid(x): return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x): return x * (1 - x)

def train_gate(X, y, lr=0.1, epochs=10000):
w, b = np.random.rand(2,1), np.random.rand(1)
for _ in range(epochs):
z = np.dot(X, w) + b
out = sigmoid(z)
error = y - out
w += lr * np.dot(X.T, error * sigmoid_derivative(out))
b += lr * np.sum(error * sigmoid_derivative(out))
return sigmoid(np.dot(X, w) + b).round()

# Inputs and labels
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y_and = np.array([[0],[0],[0],[1]])
y_or = np.array([[0],[1],[1],[1]])

print("By Shudarsan Paudel")

print("AND Gate Predictions:", train_gate(X, y_and).flatten())
print("OR Gate Predictions:", train_gate(X, y_or).flatten())
```

Output:

```
By Shudarsan Paudel
AND Gate Predictions: [0. 0. 0. 1.]
OR  Gate Predictions: [0. 1. 1. 1.]
```

 # Backpropagation Learning for XOR

```
import numpy as np
# Activation functions
def sigmoid(x): return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x): return x * (1 - x)

# Dataset (XOR)
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

# Network structure
input_neurons, hidden_neurons, output_neurons = 2, 2, 1
lr, epochs = 0.5, 10000

# Initialize weights & biases
W1 = np.random.uniform(size=(input_neurons, hidden_neurons))
W2 = np.random.uniform(size=(hidden_neurons, output_neurons))
b1 = np.random.uniform(size=(1, hidden_neurons))
b2 = np.random.uniform(size=(1, output_neurons))
```

```
# Training
for _ in range(epochs):
# Forward pass
h_in = np.dot(X, W1) + b1
h_out = sigmoid(h_in)
f_in = np.dot(h_out, W2) + b2
out = sigmoid(f_in)

# Backward pass
error = y - out
d_out = error * sigmoid_derivative(out)
d_hidden = d_out.dot(W2.T) * sigmoid_derivative(h_out)

# Update weights & biases
W2 += h_out.T.dot(d_out) * lr
b2 += np.sum(d_out, axis=0, keepdims=True) * lr
W1 += X.T.dot(d_hidden) * lr
b1 += np.sum(d_hidden, axis=0, keepdims=True) * lr

# Results
print("By Shudarsan Paudel")
print("Training complete for XOR using Backpropagation.")
print("Predicted Output:\n", out.round())
```

Output:

```
By Shudarsan Paudel
Training complete for XOR using Backpropagation.
Predicted Output:
 [[0.]
 [1.]
 [0.]
 [1.]]
```

LAB 5: Applications of AI

Weather Prediction Algorithm

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
```

```
# Dataset
data = {
'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast',
'Sunny',
'Sunny', 'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild',
'Cool', 'Mild', 'Mild', 'Mild', 'Hot', 'Mild'],
'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal',
'High',
'Normal', 'Normal', 'Normal', 'High', 'Normal', 'High'],
'Windy': ['False', 'True', 'False', 'False', 'False', 'True', 'True', 'False',
'False', 'False', 'True', 'True', 'False', 'True'],
'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No',
'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)

# Encode categorical variables
label_encoders = {}
df_encoded = df.copy() # keep original df safe

for col in df_encoded.columns:
le = LabelEncoder()
df_encoded[col] = le.fit_transform(df_encoded[col])
label_encoders[col] = le

# Split features and target
X = df_encoded.drop('Play', axis=1)
y = df_encoded['Play']

# Train Decision Tree Classifier
classifier = DecisionTreeClassifier()
classifier.fit(X, y)

# Prediction function (fixed to avoid KeyError)
def predict_weather(outlook, temperature, humidity, windy):
df_input = pd.DataFrame([{
'Outlook': outlook,
'Temperature': temperature,
'Humidity': humidity,
'Windy': windy
}])
# Encode input
for col, le in label_encoders.items():
if col != 'Play': # don't encode target column
df_input[col] = le.transform(df_input[col])
prediction = classifier.predict(df_input)[0]
return label_encoders['Play'].inverse_transform([prediction])[0]

# Test prediction
outlook, temperature, humidity, windy = 'Sunny', 'Cool', 'High', 'True'

print("By Shudarsan Paudel")
print(f"Weather conditions: Outlook={outlook}, Temperature={temperature}, Humid-
ity={humidity}, Windy={windy}")
```

```
print("Prediction:", predict_weather(outlook, temperature, humidity, windy))
```

Output:

```
Weather conditions: Outlook=Sunny, Temperature=Cool, Humidity=High, Windy=True
Prediction: No
```

Natural Language Processing Example using NLTK

```
import nltk
import os
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

# --- Download required NLTK resources ---
# Removing manual path appending to rely on NLTK's default behavior
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab')


# --- Example text ---
text = ("Natural Language Processing is a fascinating field of artificial intel-
ligence. "
"It's used in chatbots, translators, and search engines.")

# --- Sentence Tokenization ---
sentences = sent_tokenize(text)
print("By Shudarsan Paudel")
print("Sentences:")
for s in sentences:
print("-", s)

# --- Word Tokenization ---
words = word_tokenize(text)
print("\nWords:", words)

# --- Stopwords Removal ---
stop_words = set(stopwords.words("english"))
filtered_words = [w for w in words if w.lower() not in stop_words]
print("\nFiltered Words:", filtered_words)

# --- Stemming ---
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(w) for w in filtered_words]
print("\nStemmed Words:", stemmed_words)

# --- Lemmatization ---
lemmatizer = WordNetLemmatizer()
```

```
lemmatized_words = [lemmatizer.lemmatize(w) for w in filtered_words]
print("\nLemmatized Words:", lemmatized_words)

# --- Part-of-Speech (POS) Tagging ---
pos_tags = nltk.pos_tag(words)
print("\nPOS Tags:", pos_tags)
```

Output: